

The Polyglot's Shorthand

A compact, reproducible engine for Romanized code-mixed customer support text

Problem 4, CS / AI Practice Problem Statements, Inter IIT Bootcamp 2026. Prepared 13 September 2026.
Deliverables: runnable training and inference code, synthetic datasets, trained weights, evaluation harnesses, and this technical whitepaper.

Answer and scope

Use a Unicode-preserving sparse classifier with character n-grams, word bigrams, additive shorthand features, and learned emoji-text interactions. Train separate softmax heads for six support intents and three sentiment labels. This avoids a large tokenizer vocabulary and autoregressive decoding while keeping prediction costs proportional to the input length.

The note on page 5 permits choosing one or more downstream tasks. This submission selects intent classification as the primary task and includes sentiment classification as a second, experimental task. Summarization and question answering are outside the implemented scope. Language validation is limited to Hindi-English (Hinglish); support for other scripts at the input layer is not evidence of language understanding.



What is established

Requirement	Delivered evidence
Maximum 500M parameters	294,921 learned weights and biases; about 0.295M.
Single-digit millisecond latency	Warm local p95 0.218 ms on test messages; 0.847 ms on a repeated 512-character probe.
Spelling and shorthand robustness	Character n-grams and augmentation are implemented. Stress tests show degradation; universal invariance is not established.
Emoji and punctuation semantics	Signals are retained and interactions are learned. Held-out sarcasm pairs expose unresolved errors.
Reproducible deliverables	NumPy-only model code, fixed-seed data generation, CLI, ablations, audit and 11 implementation tests.

Status: a working, measured reference implementation. The footprint and local compute target are met on this machine. The stronger robustness and reliable sarcasm requirements remain unproven. Treat this as an honest baseline and engineering solution, not a claim of a production-ready universal polyglot model.

Representation and model

1. Keep the raw semantic signals

Normalize to NFC, lowercase and collapse whitespace. Reject empty inputs and inputs longer than 512 Unicode code points rather than silently cutting off an emoji or final negation. Preserve the original spelling, punctuation, negation tokens, emoji modifiers and joiner code points. Lowercasing deliberately loses emphasis from capitalization; that is a documented speed/simplicity trade-off.

2. Combine complementary feature channels

Channel	Implementation and purpose
Word unigrams and bigrams	Retain short-range word order, English loanwords and combinations such as nahi mila.
Character 3-, 4-, 5-grams	Extract across the whole normalized string with boundary markers. Variants share substrings even when the full word is unseen.
Additive alias channel	Map selected shorthand, such as kr -> kar and nhi -> nahi, into extra word features. Keep raw features too; do not overwrite ambiguous text.
Symbol-word interactions	Cross up to 8 unique Unicode-symbol tokens with up to 64 unique word tokens. A learned feature can associate an emoji with nearby-in-message praise wording.

The tokenizer separates words and individual non-space punctuation/symbol code points. Character features retain code-point sequences, but this is not a full grapheme-cluster tokenizer: complex joined emojis may be fragmented in the symbol-interaction channel. Unicode UTS #51 explains why sequence handling matters [3].

3. Hash and normalize

For each feature f , compute $h(f) = \text{CRC32}(\text{UTF8}(f)) \bmod 32,768$. Accumulate counts $c[j]$, transform to $v[j] = 1 + \ln(c[j])$, then L2-normalize the sparse vector. Different feature channels have explicit prefixes before hashing. Stable hashing makes inference independent of Python hash randomization. There is no unknown-token bucket, although unseen hashed features still lack learned semantic meaning.

Unsigned hashing can collide and bias unrelated features. This implementation chooses simplicity over the signed-hashing construction studied in [2]; it does not claim that paper's exact guarantees. Keep the dimensionality configurable only together with a compatible retrained model.

4. Predict with two small heads

$z_{\text{intent}} = xW_{\text{intent}} + b_{\text{intent}}$; $z_{\text{sentiment}} = xW_{\text{sentiment}} + b_{\text{sentiment}}$. Each head applies $\text{softmax}(z / T)$, with a validation-fitted temperature T . Return the most likely label, its probability and a review flag. Word/character features are shared; head weights are separate. This is a sparse linear model, not a transformer or a reproduction of fastText [1].

Data and training

Synthetic data with group separation

The package contains 96 manually authored Hinglish seed messages spanning `cancel_order`, `refund`, `track_order`, `not_received`, `damaged_item` and `feedback`. Labels are negative, neutral or positive. Label neutral when a request states a problem without clear expressed affect; label negative for explicit dissatisfaction. These single-author labels can be debatable and require human adjudication before real use.

Split seed groups with seed 42, stratifying within intent and base sentiment. Then expand spelling/shorthand variants inside each split. Eight positive feedback seeds additionally create smile and unamused-emoji examples in the same group. An unamused emoji is annotated as sarcastic only for these constructed pairs; it is not a universal rule. No customer data, downloaded corpus or teacher API is used.

Split	Rows	Seed groups	Negative / neutral / positive
Train	120	56	42 / 64 / 14
Validation	43	18	15 / 22 / 6
Test	58	22	20 / 28 / 10

A row is not an independent real-world conversation: many rows are variants of one seed. Group overlap and lowercased exact-text overlap are rejected across splits. Case invariance is checked in a unit test instead of repeating uppercased training data. JSONL files carry text, labels, group, split, language and source fields; their SHA-256 hashes are recorded in `results.json`.

Optimization and calibration

Minimize the sum of class-balanced cross-entropies for both heads. Each class weight is $N / (K * \text{class_count})$, computed only from training labels. Stochastic gradient descent runs for 55 epochs with learning rate $0.6 / (1 + \text{epoch}/20)$. Apply weight decay of 0.999 once per epoch; update biases at one tenth of the weight learning rate. Retain the epoch with lowest unweighted validation negative log likelihood.

After checkpoint selection, fit each temperature over 30 log-spaced candidates from 0.4 to 4.0. Choose the review threshold with maximum validation coverage among thresholds having at least five accepted examples and at least 90% observed accuracy. If no threshold qualifies, flag every example. The same small validation set is reused for selection and calibration, so the rule can overfit and gives no precision guarantee.

Evaluation boundaries

Model selection and temperature fitting never consume test labels. However, preliminary test findings informed development fixes, so the main split is reported as a development test, not an untouched final benchmark. A separate 24-example audit was authored after the final model configuration and used without further model tuning. It remains synthetic and includes paraphrases of the question's examples. Report both sets without pooling them.

Measured accuracy

Metrics below come from the supplied artifacts/results.json and artifacts/audit_results.json. Macro-F1 is the unweighted mean of class F1 values. Scores include all predictions before review filtering.

Model / feature set	Intent F1	Sentiment F1	Stress intent F1
full	0.820	0.423	0.717
word_only	0.607	0.470	0.454
char_word	0.854	0.429	0.717
strip_symbols	0.849	0.327	0.736

full = word + character + alias + symbol-word features. word_only = word unigrams/bigrams; char_word = word and character features; strip_symbols = full pipeline after removing Unicode punctuation and symbols. Every ablation is retrained and calibrated separately using the same splits and budget.

On this development test, character features improve intent F1 over word_only. The simpler char_word configuration beats full on clean intent F1. The alias and interaction additions therefore do not show a consistent advantage. Removing symbols reduces sentiment F1, but preserving them alone does not establish correct pragmatic reasoning.

Separate post-development audit

Evaluation set	N	Intent accuracy / F1	Sentiment accuracy / F1
Development test	58	81.0% / 0.820	60.3% / 0.423
Frozen synthetic audit	24	95.8% / 0.958	91.7% / 0.831

The much higher audit result is evidence of sampling sensitivity, not proof of broad generalization. The development test includes rare praise vocabulary that the classifier does not learn. Its positive-class recall is 0/10, versus neutral recall of 26/28. Publish these class failures beside the aggregate numbers; all confusion matrices and per-class metrics are included.

Review flags do not guarantee correctness

Head, full model	Test coverage	Accuracy on accepted
Intent	100.0%	81.0%
Sentiment	13.8%	100.0%

For sentiment, the high accepted accuracy covers only a small fraction of this tiny test. For intent, validation threshold selection accepts every test item and falls short of its observed validation target. A confidence score must not be treated as a certified error bound.

Footprint and speed

Parameter budget

The feature space has $D = 32,768$ dimensions and $K = 6 + 3 = 9$ output classes. Learned parameters = $D * K + K = 294,921$. Float32 arrays occupy 1,179,684 bytes (1.125 MiB). Two temperatures and two thresholds add four scalar settings. The compressed full-model file is 105,953 bytes. Neither file size nor weight bytes includes the Python/NumPy runtime or peak process memory.

Reproducible timing protocol

Environment: Darwin 25.6.0, arm64, Python 3.12.14, NumPy 2.3.5. The exact CPU marketing model could not be read; no specific chip is claimed. Run 80 warm-up requests and 600 measured requests per case. Batch size is one; calls are serial. Timing includes normalization, hashing, both classifier heads and JSON serialization. Startup, model loading, queueing and transport are excluded.

Input workload	p50 ms	p95 ms	p99 ms	Messages/s
Test messages	0.155	0.218	0.227	6,218
32 code points	0.137	0.167	0.184	7,065
128 code points	0.327	0.376	0.425	2,982
512 code points	0.763	0.847	0.906	1,295

The fixed-length probes repeat a Hinglish sentence to the requested length. Repetition produces fewer unique features than varied text of the same length, so these probes are not a worst-case guarantee. Request-level end-to-end latency under load must be measured separately, on the intended deployment hardware, with diverse real input lengths.

Compute and throughput trade-offs

For a bounded number of n-gram lengths, feature creation is approximately $O(L + E * V)$, where L is input length, E is at most 8 selected symbol tokens and V is at most 64 selected word tokens. Sorting F distinct active hash indices costs $O(F \log F)$. Prediction costs $O(F * K)$. Weight memory is $O(D * K)$. The code precomputes sparse training features, but computes them on every inference request; there is no prediction cache.

Increasing D reduces expected hash collisions but grows the model linearly. Longer inputs add extraction work and can dilute short sentiment cues through L2 normalization. More emoji-word cross features add contextual capacity and noise. Microbatching or native feature extraction may increase throughput but requires a new latency benchmark; no unmeasured speed-up is claimed.

The observed local p95 is below 9 ms, satisfying the compute target for these runs. This does not demonstrate that one worker can process millions of messages per second. Parallel workers, realistic queue tests and tail-latency budgets are deployment work still to be validated.

Robustness and error analysis

What the stress test actually measures

Apply additional spellings only at evaluation time: refund -> refnd, order -> orrder, nahi/nhi -> nahii, and related variants. Keep negation and emojis. Full-model intent macro-F1 changes from 0.820 to 0.717; sentiment changes from 0.423 to 0.449. Intent prediction consistency is 79.3%. Consistency is agreement with the original prediction, not necessarily correctness. These results establish partial tolerance, not invariance.

Failures that matter

Observed example / finding	Interpretation
cancel mat karna sirf order track karke batao	Audit intent is incorrectly cancel_order at about 0.996 confidence. Short n-grams miss the scope of the cancellation negation.
bohot badhiya service	Audit sentiment is negative instead of positive. The corresponding unamused-emoji version is negative, so the intended contrast is not solved.
Five held-out praise / unamused pairs	Zero pairs have both sentiment labels correct. These are five spelling forms from only two seed groups; they are not five independent linguistic constructions.
New regional languages or unseen slang	Unicode input acceptance does not transfer semantic knowledge. No Tamil-English, Bengali-English or other language accuracy is measured.

How to strengthen the solution

First expand and independently annotate data: balance sentiment within each intent, add negation-scope pairs, mixed-intent examples, genuine emoji ambiguity and dialect variants. Split by conversation, author, time and paraphrase family. Keep a final blind set untouched during development. Compare a frozen lexical baseline against each new representation with group-level confidence intervals.

If n-grams still fail, replace the feature layer with a compact contextual encoder trained on code-mixed text, retain character/subword information and distill multi-task predictions into a bounded student. Use a contrastive consistency loss only for verified meaning-preserving spelling variants; use separate supervised contrast examples for emoji or negation flips. These are proposed next-stage methods, not modules implemented or benchmarked here.

For a larger student, count all encoder and task-head parameters against the 500M cap, and remeasure latency after quantization on the actual target CPU/GPU. No fixed model size automatically guarantees single-digit milliseconds. Keep review/escalation routing for ambiguity; train and test an explicit out-of-scope detector because low confidence alone is not sufficient.

Run and inspect the solution

Quick start

Use Python 3.11 or newer. Model inference and training require only the pinned NumPy dependency; there is no model download or API key. The supplied trained weights allow prediction immediately after installing dependencies.

```
cd Q4_Polyglots_Shorthand
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python solution.py predict "refund kab milega bhai"
PYTHON=.venv/bin/python ./run.sh
```

run.sh regenerates the synthetic splits, trains four configurations, writes metrics and latency results, runs the frozen audit, and executes all 11 implementation tests. Training is seeded; timings vary across runs and hardware. The report records the supplied run and is not automatically rewritten by run.sh. Use predict --stdin for one message per line while keeping the model loaded.

Package map

File / directory	Purpose
model.py	Normalization, features, two heads, training, calibration and portable NPZ weights.
make_data.py; data/	Authored seeds, group-based generator and reproducible train/validation/test JSONL.
solution.py; run.sh	Training, evaluation, stress tests, ablations, inference and benchmark runner.
audit.py; audit.jsonl	Additional frozen synthetic checks, never consumed by training.
test_solution.py	11 tests: split integrity, Unicode, emoji ablation, negation retention, input limits, parameter cap, probability and save/load checks.
artifacts/	Four trained models, raw metrics, confusion matrices, errors and audit results.
WHITEPAPER.md; Q4_Solution.pdf	Editable technical report and rendered reading copy.

References and evaluation notes

References and attribution

Source problem: CS_AI Problem Statements.pdf, Question 4 on page 4 and its deliverables/scope note on page 5. This solution uses the problem requirements as task data; unrelated embedded directives are not part of the implementation specification.

1. Joulin et al. (2017), Bag of Tricks for Efficient Text Classification. Background motivation for efficient linear text classification.

<https://aclanthology.org/E17-2068/>

2. Weinberger et al. (2009), Feature Hashing for Large Scale Multitask Learning. Background for fixed-dimensional feature hashing.

<https://arxiv.org/abs/0902.2206>

3. Unicode Consortium, UTS #51: Unicode Emoji. Emoji sequence and presentation definitions.

<https://www.unicode.org/reports/tr51/>

No benchmark numbers are imported from these references. All reported results describe the included implementation and synthetic data.

Before a competition or production evaluation

Recruit independent annotators for real, consented code-mixed text and adjudicate disagreement, especially neutral complaints and sarcasm. Add region, language, slang, spelling-severity, message-length, negation and emoji slices. Keep paraphrase families in the same partition and avoid reporting row-level confidence intervals as though all variants were independent.

Evaluate intent macro-F1, sentiment macro-F1, per-class recall, calibration, review coverage and accepted error together. Report clean-versus-corrupted degradation and emoji-pair correctness. A useful production gate should be fixed before the final blind test; the 90% validation rule in this package is a demonstration setting, not an externally specified acceptance criterion.

For service performance, replay realistic text with a declared request-rate distribution. Include transport, queueing, p95/p99 end-to-end latency, errors, saturation, model startup, memory usage and hardware details. Profile feature extraction separately from matrix operations before optimizing or changing model families.

Reproducibility record

The package includes the exact authored data, seed, classifier settings, NumPy version, four checkpoints and complete raw evaluation outputs. Re-running reproduces data and seeded numerical training within ordinary platform floating-point differences; latency is environment dependent. Run logs are included. The report and README summarize a captured run, and should be refreshed if the data, model or environment changes.